SREENIDHI EDUCATIONAL GROUP | SREENIDHI INSTITUTE OF SCIENCE AND TECHNOLOGY AUTONOMOUS

COURSE
ARTIFICIAL INTELLIGENCE AND DEEP LEARNING (AIDL)

UNIT-VI

Mr. P Ramu,Assistant Professor

DEEP REINFORCEMENT LEARNING:

1. Deep Reinforcement Learning Masters Atari Games
2. Reinforcement Learning
3. Markov Decision Processes (MDP)
4. Explore Versus Exploit
5. Pole Cart with Policy Gradients
6. Q - Learning and Deep Q-Networks
7. Improving and Moving Beyond DQN.

Deep Reinforcement Learning

Definition:

1. Reinforcement learning, which is a branch of machine learning that deals with learning via interaction and feedback.

2. Reinforcement learning is essential to building agents that can not only perceive and interpret the world, but also take action and interact with it.

3. Discuss how to incorporate deep neural networks into the framework of reinforcement learning and discuss recent advances and improvements in this field.

What Is Reinforcement Learning?

- Reinforcement learning, at its essentials, is learning by interacting with an environment.
- Learning process involves an *actor, an environment, and a reward signal.*
- The actor chooses to take an action in the environment, for which the actor is rewarded accordingly.
- The way in which an actor chooses actions is called a *policy.*
- *The* actor wants to increase the reward it receives, and so must learn an optimal policy for interacting with the environment (Figure 9-2).
- Reinforcement learning is different from the other types of learning that we have covered thus far.
- In traditional supervised learning, we are given data and labels, and are tasked with predicting labels given data.

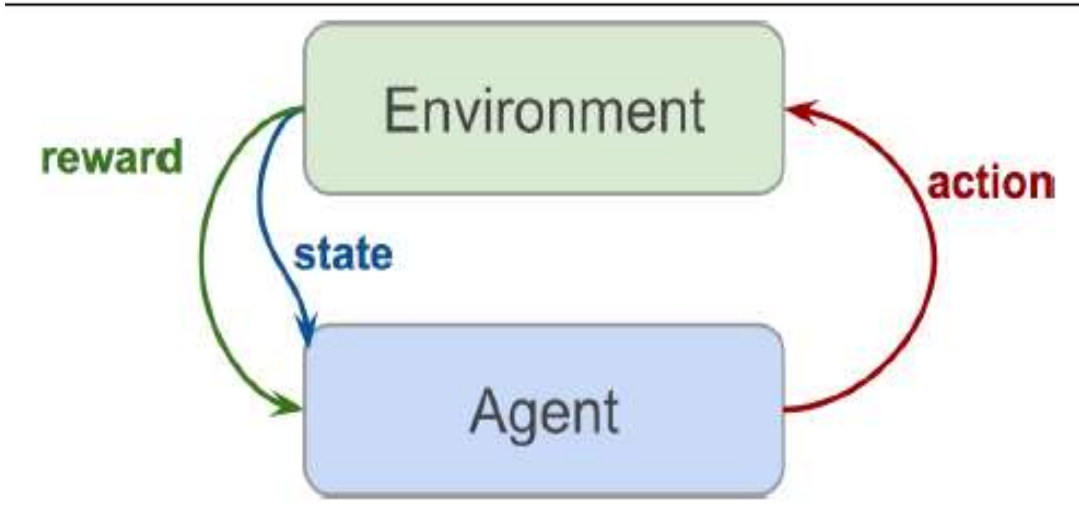


Figure 9-2. Reinforcement learning setup

- In unsupervised learning, we are given just data and are tasked with discovering underlying structure in this data.
- In reinforcement learning, we are given neither data nor labels.
- Learning signal is derived from the rewards given to the agent by the environment.

1. Deep Reinforcement Learning Masters Atari Games

1. Introduction
2. Problem Statement
3. Key Idea
4. Reinforcement Learning Setup
5. Deep Q-Network (DQN)
6. Key Techniques Used
7. Training Process
8. Achievements
9. Importance
10. Limitations
11. Improvements Beyond DQN

1. Introduction

A major breakthrough in AI occurred when **DeepMind** developed a system that could **learn to play Atari video games directly from raw pixels**, achieving **human-level performance**.
This approach combines **Reinforcement Learning (RL)** with Deep Learning, known as **Deep Reinforcement Learning (DRL)**.

2. Problem Statement

- Input: Raw game frames (pixel images)
- Output: Game actions (left, right, fire, etc.)
- Goal: Maximize total score

->No manual feature extraction is used.

3. Key Idea

- Use a **Deep Q-Network (DQN)** to:
- Learn directly from images
- Estimate **Q-values** for actions
- Select optimal actions

4. Reinforcement Learning Setup

- **Agent:** Game player
- **Environment:** Atari game
- **State:** Current screen (image)
- **Action:** Game controls
- **Reward:** Score change

Objective: Maximize cumulative reward.

- **5. Deep Q-Network (DQN)**
- **Concept**
- DQN combines:
- Q-Learning (RL)
- Deep Neural Networks
- **Input**
- Sequence of frames
- **Output**
- Q-values for each action

6. Key Techniques Used

1. Experience Replay

- Stores past experiences (s, a, r, s')
- Random sampling for training
- ☞ Improves learning stability

2. Target Network

- Separate network for target Q-values
- Updated periodically

☞ Prevents instability

3. Reward System

- Positive reward → good action
- Negative reward → bad action

7. Training Process

- Observe game state
- Choose action (ε-greedy)
- Execute action
- Receive reward
- Store experience
- Train neural network
- Update Q-values

8. Achievements

- Played **49 Atari games**
- Achieved **human-level performance**
- Learned strategies like:
  - Paddle control in Pong
  - Brick breaking in Breakout

9. Importance

- First system to learn from **raw pixels**
- No domain-specific knowledge required
- Same model works across multiple games
- ☞ Major milestone in AI

10. Limitations

- High training time
- Requires GPUs
- Performance varies across games

11. Improvements Beyond DQN

- **Double DQN:** Reduces overestimation
- **Dueling DQN:** Separates value & advantage
- **Prioritized Replay:** Focus on important data
- **Actor-Critic:** Combines policy & value methods

2. Reinforcement Learning

1. Definition

Reinforcement Learning (RL) is a type of machine learning where an **agent learns by interacting with an environment** and improves its actions based on rewards or penalties.
It is a key concept in Artificial Intelligence.

2. Basic Components

- **Agent** → Learner/decision maker
- **Environment** → External system
- **State (S)** → Current situation
- **Action (A)** → Choice made by agent
- **Reward (R)** → Feedback signal
- **Policy (π)** → Strategy for selecting actions

3. Working of RL

- Agent → Action → Environment → Reward → New State → Agent

Goal: **Maximize cumulative reward**

4. Key Concepts

Reward

- Immediate feedback after action

Return

- Total accumulated reward

Discount Factor (γ)

- Importance of future rewards

3. Markov Decision Processes (MDP)

1. Definition

A **Markov Decision Process (MDP)** is a mathematical framework used to model **decision-making in situations where outcomes are partly random and partly under the control of an agent**.
It is widely used in Artificial Intelligence and reinforcement learning.

2. Components of MDP

An MDP is defined as:

- (S, A, P, R, γ)
- **S (States):** All possible situations
- **A (Actions):** Choices available to the agent
- **P (Transition Probability):** Probability of moving from one state to another
- **R (Reward):** Immediate feedback
- **γ (Gamma):** Discount factor ($0 \leq \gamma \leq 1$)

3. Markov Property

Future state depends only on the present state, not on past

$$P(S_{t+1}|S_t, S_{t-1}, \ldots) = P(S_{t+1}|S_t)$$

4. Working of MDP

- Current State → Action → Next State → Reward

The agent selects actions to **maximize total reward** over time.

5. Policy (π)

- A **policy** defines the agent's behavior
- Maps states → actions

Optimal policy = best strategy for maximum reward

6. Value Functions

- **(a) State Value Function**
- $V(s) = Expected\ reward\ from\ state$
- **(b) Action Value Function (Q-function)**
- $Q(s, a)$
  $= Expected\ reward\ for\ taking\ action\ a\ in\ state\ s$

7. Bellman Equation

- $V(s) = \max_a \quad \sum_{s'} P\,(s', \mid sa)[R + \gamma V(s')]$

Used to compute optimal values.

8. Solution Methods

- **1. Value Iteration**
- Repeatedly update values
- Converges to optimal solution
- **2. Policy Iteration**
- Policy evaluation
- Policy improvement

9. Example

Robot navigation:

States → Locations

Actions → Move directions

Reward → Goal reached

10. Applications

- Robotics
- Game playing
- Self-driving cars
- **11. Advantages**
- Handles uncertainty
- Provides optimal decisions
- Widely applicable

12. Limitations

- Large state space problem
- Requires known probabilities

4. Explore Versus Exploit

- **1. Definition**
- The **Explore vs Exploit trade-off** is a key concept in reinforcement learning where an agent must choose between:
- **Exploration** → Trying new actions to discover better rewards
- **Exploitation** → Using known actions that already give high rewards
- ☞ The challenge is to **balance both** to maximize long-term reward.

2. Explanation

Exploration

- Tries unknown actions
- Gains new knowledge
- May give low reward initially

Example: Trying a new path in a maze

Exploitation

- Uses best-known action
- Gives immediate high reward

Example: Repeating the shortest known path

3. Trade-off Problem

- Too much **exploration** → Wastes time, low reward
- Too much **exploitation** → May miss better solutions

Need a balance for optimal learning.

4. Methods to Balance

1. ε-Greedy Strategy
- With probability **ε** → explore
- With probability **1–ε** → exploit

2. Decaying ε
- Start with high exploration
- Gradually reduce exploration

3. Softmax (Boltzmann)
- Select actions based on probability
- Better actions chosen more often

4. Upper Confidence Bound (UCB)
- Balances reward and uncertainty

5. Example

In a game:

- Exploration → Try new moves
- Exploitation → Use best move found

6. Importance

- Ensures learning of optimal policy
- Avoids local optimum
- Improves long-term performance

7. Applications

- Game playing (Atari, Chess)
- Robotics
- Recommendation systems

5. Pole Cart with Policy Gradients

1. Introduction

The **Pole–Cart (CartPole)** problem is a classic control task in reinforcement learning where an agent must **balance a pole upright on a moving cart**.
It is widely used to demonstrate **policy-based methods**, especially **Policy Gradient algorithms**.

2. Problem Description

- A cart can move **left or right** on a track
- A pole is attached and tends to fall due to gravity
- The agent must keep the pole **balanced (upright)**

3. RL Formulation

State (S):

- Cart position
- Cart velocity
- Pole angle
- Pole angular velocity

Action (A):

- Move left
- Move right

Reward (R):

- +1 for every time step the pole remains upright

Goal: **Maximize total reward (longer balance time)**

4. Why Policy Gradients?

- Instead of learning value functions (like Q-learning),
- Policy gradients **directly learn the policy**:
- $\pi(a; | s\theta)$

Output: Probability of taking action **a** in state **s**

- **5. Policy Gradient Idea**
- Represent policy using a neural network
- Update parameters **θ** to maximize reward
- **Objective:**
- $J(\theta) = \mathbb{E}[R]$
- **Update Rule:**
- $\theta = \theta + \alpha \nabla J(\theta)$

6. Working Steps

- Observe current state
- Choose action based on probability (policy)
- Execute action
- Receive reward
- Store trajectory (states, actions, rewards)
- Update policy using gradient ascent

7. Algorithm (Simple)

1. Initialize policy parameters
2. Repeat:
   - Generate episode
   - Compute rewards
   - Update policy
3. Until pole stays balanced

8. Advantages

- Works well in continuous action spaces
- Learns stochastic policies
- No need to store Q-values

9. Limitations

- High variance in learning
- Slow convergence
- Sensitive to hyperparameters

10. Applications

- Robotics control
- Game AI
- Autonomous systems

6. Q - Learning and Deep Q-Networks

- **1. Introduction**
- **Q-Learning** is a model-free reinforcement learning algorithm that learns the **optimal action-value function**.
  When combined with neural networks, it becomes a **Deep Q-Network (DQN)**, enabling learning from high-dimensional inputs like images.
  These methods are central to Artificial Intelligence and modern reinforcement learning.

- **2. Q-Learning**
- **◆ Definition**
- Q-Learning learns the value of taking an action in a given state:
- $Q(s, a)$
- ☞ Goal: Find optimal policy that maximizes cumulative reward.

◆ **Update Rule (Bellman Equation)**

$$Q(s,a) = Q(s,a) + \alpha\left[R + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

Where:

- α = learning rate
- γ = discount factor
- R = reward
- s' = next state

Working Steps

- Initialize Q-table
- Observe state
- Choose action (ε-greedy)
- Execute action
- Update Q-value
- Repeat

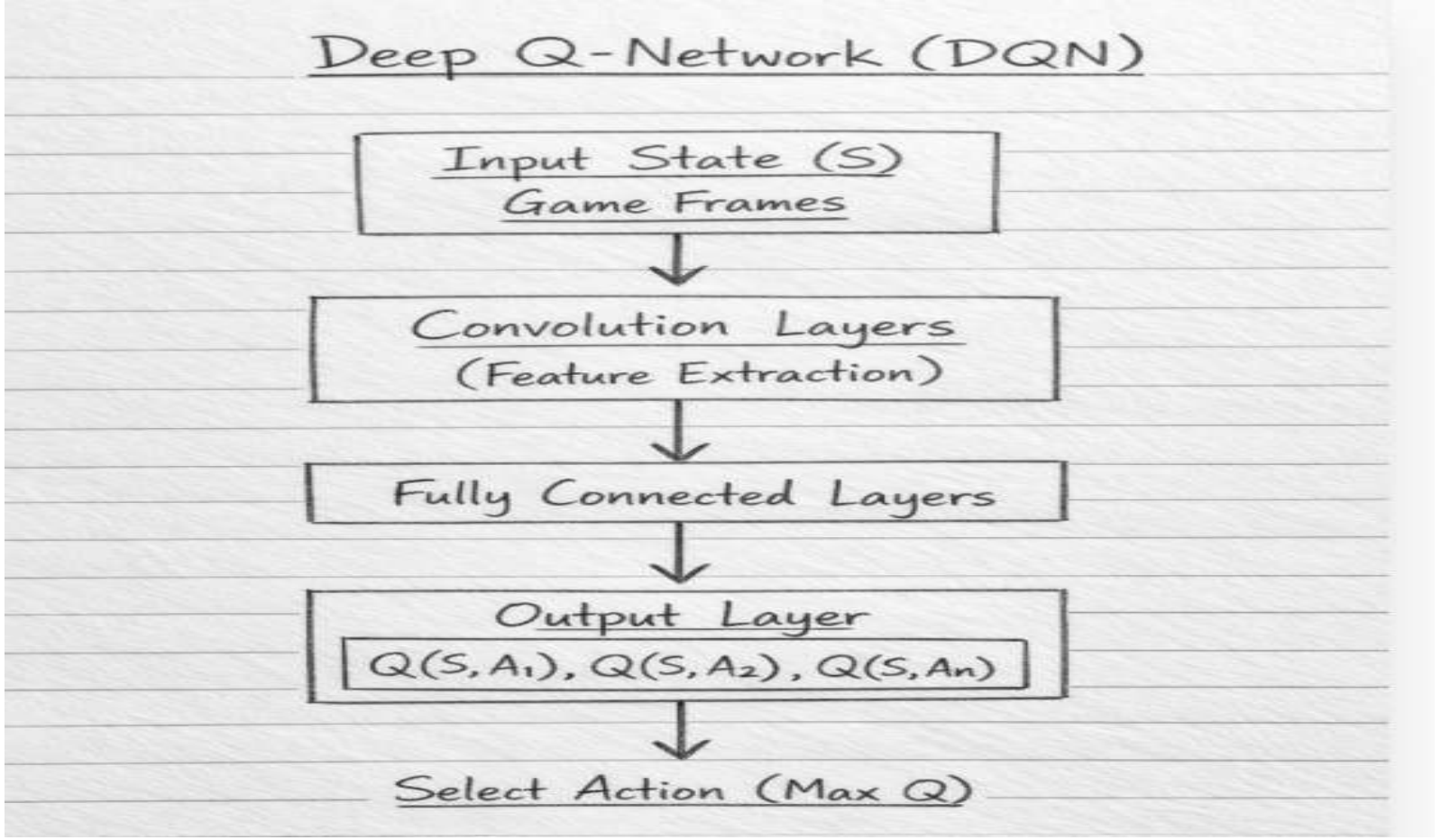
Advantages

- Simple and effective
- Learns optimal policy
- No model required

Limitations

- Not scalable to large state spaces
- Requires Q-table (memory issue)

- **3. Deep Q-Network (DQN)**
- **◆ Definition**
- DQN replaces the Q-table with a **deep neural network** to approximate Q-values.



Key Techniques

- **1. Experience Replay**
- Stores past experiences
- Random sampling for training
- **2. Target Network**
- Separate network for stable learning

Working Steps

- Observe state
- Select action (ε-greedy)
- Execute action
- Store experience (s,a,r,s')
- Sample mini-batch
- Train neural network
- Update target network

Advantages

- Handles large state spaces
- Works with images
- Learns complex patterns

Limitations

- Requires high computation
- Training instability
- Needs large data

Applications

- Game playing (Atari)
- Robotics
- Autonomous vehicles

Q-Learning vs DQN

Feature	Q-Learning	DQN
Representation	Q-table	Neural Network
State Space	Small	Large
Input	Discrete	High-dimensional
Performance	Limited	High

7. Improving and Moving Beyond DQN

1. Introduction

- While **Deep Q-Networks (DQN)** achieved great success (e.g., Atari games by **DeepMind**), they have limitations such as **overestimation, instability, and inefficient learning**.
  To overcome these issues, several improved algorithms were developed.
  2. Limitations of DQN
- Overestimation of Q-values
- Slow learning
- Correlated data issues
- Poor sample efficiency
- Instability during training

3. Improvements Over DQN

1. Double DQN (DDQN)

Problem: DQN **overestimates Q-values**

- **Solution:**
- Uses two networks:
  - One for action selection
  - One for evaluation
- ✓ Reduces overestimation
  ✓ Improves accuracy

2. Dueling DQN

Idea: Separate learning into:

- **State Value V(s)**
- **Advantage A(s,a)**
- $Q(s, a) = V(s) + A(s, a)$
- ✓Better learning of important states
  ✓ Faster convergence

3. Prioritized Experience Replay

Problem: Random sampling treats all experiences equally

- **Solution:**
- Prioritize important experiences (high error)
- ✓ Learns faster
  ✓ Improves efficiency

4. Noisy Networks

Problem: Exploration using ε-greedy is inefficient

Solution:

- Add noise to network weights
- ✓ Better exploration
  ✓ No need for ε-greedy

5. Multi-Step Learning

Instead of one-step reward:

- Uses multiple future rewards
- ✓ Faster learning
  ✓ Better reward propagation

4. Advanced Methods Beyond DQN

1. Policy Gradient Methods

- Directly learn policy
- Suitable for continuous actions

2. Actor-Critic Methods

Combines:

- **Actor:** Selects action
- **Critic:** Evaluates action
- ✓ Stable learning
  ✓ Faster convergence

3. A3C (Asynchronous Advantage Actor-Critic)

- Multiple agents learn in parallel
- Faster and more stable

4. PPO (Proximal Policy Optimization)

- Limits large updates
- Stable and widely used

5. Comparison Table

Method	Improvement
Double DQN	Reduces overestimation
Dueling DQN	Better state evaluation
Prioritized Replay	Faster learning
Actor-Critic	Stability & performance
PPO	Safe updates